Abducer: Contract-Enforced Authority Separation for LLM-Based Abductive Workflows

Neuro-symbolic AI Group
Idiap Research Institute

August 2026

Abstract

Large-language-model workflows can generate scientific hypotheses, but generation alone does not explain how a proposal becomes durable scientific state, how rival explanations remain plural, or how preference is prevented from silently becoming proof or empirical support. We describe Abducer as an *authority-separated abductive transition system* (AATS). Its abductive kernel constructs and transforms competing conceptual systems, exposes missing premises, preserves underdetermination through set-valued adjudication, and uses negative results to constrain later search. Its accountability shell binds operator artifacts to exact identities, semantic consumer events, capability-specific evidence, deterministic reduction, and an atomic state transition. Around this kernel, named workflows provide trigger-governed escalation lanes for target control, source assimilation, conceptual deepening, proof delivery, contribution assessment, manuscript elaboration, deep review, and release without transferring epistemic authority between them. Upstream operators produce a frozen artifact bundle; the canonical cycle validates and semantically consumes that bundle rather than executing the operators itself. Formal-claim packets obtain proof-launch eligibility only after committed ancestry is reopened, while experimental descendants at transaction generations N through $N+8$ have no direct edge into the theory that commissioned them. We contribute an evidence-grounded systems abstraction and conformance analysis, not evidence of scientific efficacy, theorem correctness, autonomous creativity, or novelty relative to prior systems. The integrated release surface is presently unaccepted because its checked-in publication pointer and historical acceptance topology lag the current contracts.

1 Motivation

Scientific abduction asks which conceptual system, missing premise, mechanism, or representation would make a difficult body of observations and obligations intelligible. A language model can propose such objects, but an accountable discovery process must answer harder questions: Which version of the problem did the proposal address? Which literature was available? Which rival programmes were considered? Did an evaluator prefer one programme, or did a proof establish a claim? Did a generated artifact actually affect later state? Can a failed route disappear without explanation? Can a stale worker overwrite a newer theory?

Abducer treats these as state-transition and authority questions rather than prompt-engineering details. Its stronger discovery surface is not a linear chain of agents. It is an event-sourced control system in which plural generative operators propose changes, typed validators license transport, an authority-separated evaluator returns a set-valued preference, and exactly one reducer may write canonical state.

We make three contributions:

1. a typed abductive state-and-event model that preserves rival explanations, genealogy, negative evidence, stale branches, and retired routes;

2. a capability-indexed authority model and descendant-lane protocol that keep conceptual preference, canonical state write, checker results, empirical observations, source truth, theorem status, and publication integrity mutually non-escalating; and

3. an executable realization with digest-bound semantic consumption, deterministic reduction, durable compare-and-swap commits, and adversarial conformance tests; and

4. a portfolio model for composing lightweight, opt-in, conditional, and target-specific workflows through declared handoffs and inspectable control hooks.

The first contribution is the abductive kernel; the latter contributions form its accountability and composition shell. The repository also contains an older KB-centred loop whose abduction and deduction steps remain manual prompt checkpoints. The named evidence onramp declaratively composes knowledge-preparation skills, but does not make those older checkpoints executable; we keep that interaction protocol outside the canonical systems model below.

2 Problem Formulation

Let a discovery process operate at generation g on a target τ and run ρ. The objective is not to maximize an unconstrained language-model score. It is to construct a controlled transition

$$S_g \xrightarrow{\Delta_g} S_{g+1}$$

such that every accepted semantic change is bound to the exact input generation, source artifacts, operator receipt, consumer event, authority class, and durable commit.

We formulate Abducer as

$$\mathcal{W} = (S, \mathcal{O}, C, J, Q, \mathcal{G}, T, \Gamma),$$

where S is epistemic state, $\mathcal{O}$ typed operators, C causal and identity constraints, J plural adjudication, Q the information scheduler, $\mathcal{G}$ the logical event ledger, T the atomic transaction mechanism, and Γ a set of authority capabilities. This separates three questions that are often collapsed:

1. *What was generated or executed?* — operator receipts and artifacts;

2. *What changed the system's scientific memory?* — typed consumer events and feedback edges;

3. *What may be asserted?* — conceptual, proof, empirical, source, and publication authority.

3 Typed Epistemic State

Definition 1 (Discovery state)**.** *The following tuple is a paper-level projection of the schema:*

$$S_g = (\mathcal{C}_g, \mathcal{D}_g, \mathcal{H}_g, \mathcal{E}_g, \Lambda_g, \mathcal{A}_g, \mathcal{B}_g, \mathcal{X}_g, \mathcal{Y}_g, \mathcal{U}_g, \mathcal{R}_g, \mathcal{Z}_g; \mu_g),$$

containing conceptual systems $\mathcal{C}$, concepts $\mathcal{D}$, hypotheses $\mathcal{H}$, evidence $\mathcal{E}$, literature snapshots Λ, branch attempts $\mathcal{A}$, branches $\mathcal{B}$, transformations $\mathcal{X}$, syntheses $\mathcal{Y}$, understanding deltas $\mathcal{U}$, typed relations $\mathcal{R}$, retired routes $\mathcal{Z}$, and metadata μ. Optional schema collections include predictions, proof obligations, and reflections.

Metadata μ_g records generation, event count, predecessor hash, all active non-retired systems, current literature identity, stale branches, rebase history, and semantic deduplication keys. Negative memory is a derived view across evidence relations, refuted hypotheses, failed attempts, stale branches, and retired routes; it is not one invented state map.

State names such as `latest`, `current`, or global aliases are insufficient authority because they do not bind target, run, and generation. Canonical transitions instead reopen an exact immutable predecessor and use its generation in compare-and-swap.

4 The Abductive Macro-Step

Let $P_g = (S_g, m_g)$ be the exact verified predecessor and commit manifest. Let B_g be a frozen bundle of independently produced operator requests, receipts, output descriptors, validator authority, and literature projections. Define

$$\mathsf{Validate}(P_g, B_g) = (E_g, F_g)$$

only when namespace equality, digest closure, predecessor binding, role cardinality, authority compatibility, and semantic-consumption obligations hold. Here E_g is the ordered event suffix and F_g the feedback-edge set. The canonical transition is the partial function

$$\delta(P_g, B_g) = \mathsf{CAS}_g(\mathsf{Reduce}(S_g, E_g), E_g, F_g, B_g)\,.$$

Generative operators construct B_g through a conditional DAG; they do not acquire canonical state-write capability. Only validation, reduction, and a successful compare-and-swap produce S_{g+1}.

$$\begin{gathered} S_g + \text{frozen literature} \longrightarrow \text{divergent proposals} \longrightarrow \text{isolated maturation} \\ \longrightarrow \text{reflection and plural adjudication} \longrightarrow B_g \\ B_g \longrightarrow \text{validation} \longrightarrow (E_g, F_g) \longrightarrow \text{reducer/CAS} \longrightarrow S_{g+1} \\ S_{g+1} \dashrightarrow \text{proof-v3 and descendant experiments (no direct back-edge into } S_g) \end{gathered}$$

Figure 1: The abductive kernel constructs a frozen bundle; the accountability shell alone commits canonical state.

4.1 Literature freezing and causal rebase

The literature operator consumes a target-scoped, versioned search snapshot and an immutable production bundle. A rebase adds new interpretation and marks affected branches stale; it does not rewrite the pre-search intuition. Novelty and investment claims are unavailable when search provenance or freshness is absent. This makes literature causality explicit: later operators must bind the committed literature projection rather than merely include a bibliography.

4.2 Divergent theory formation

The high-risk forge deliberately expands structurally different conceptual corners. Candidates carry the proposed ontology change, exact mechanism, assumptions risked, value if correct, learning if wrong, correction conditions, boundary cases, and relation to literature. Early selection is not a lexical novelty score.

4.3 Isolated parallel maturation

Candidate programmes mature through an API that deep-copies inputs and gives workers no canonical-state capability. Partial failure does not erase successful siblings, and disagreement is retained for the reducer. This is a capability boundary of the parallel API, not a claim of operating-system sandboxing.

4.4 Recursive reflection

Reflection compares programmes through material transformations: differentiating concepts, changing representations, exposing contradictions, splitting overloaded objects, or synthesizing while preserving differences. Validation requires at least two rounds. A round marked mutated, synthesized, or demoted must name structural change; two no-change rounds require an explicit plateau whose action is reframe. Renaming and unification by erasure do not count as conceptual progress.

4.5 Blind set-valued adjudication

Adjudication has three distinct layers. A structural G0–G6 evaluator checks typed transition properties. A semantic panel scores essential relevance, explanatory mediation, conceptual depth, unification power, preserved difference, generativity, reflective resilience, and truth-aptness under noncompensatory adequate floors. A blind v3 reducer sees opaque aliases and content digests rather than identities, lifecycle labels, previous preference markers, scores, or self-assessments; it returns the Pareto-nondominated eligible intersection or `revise_all`. None of these layers grants theorem authority, and the local panel is not evidence of independent human judgment.

4.6 Information scheduling

For each surviving programme, the scheduler constructs a multi-coordinate decision vector covering relevance, tension contact, understanding gain, unification, fertility, programme impact, maturity, cost efficiency, and closure safety. Deterministic Pareto layers replace one scalar objective. Protected wildcards may survive outside the first front, while a sealed plural preference converts premature synthesis into differentiation.

The next action depends on state. Semantic repetition triggers reframe and eventually retirement. Proof-ready claims may enter the proof lane. A support probe requires a mature, formalized theoretical question. Otherwise the system may formalize, differentiate, synthesize, read, or reflect. Independent actions form parallel waves; hard gates remain serial.

5 Named Workflows as Typed Escalation Lanes

The abductive macro-step defines the canonical state transition, but not every scientific request requires the same depth, cost, or authority. Abducer therefore organizes recurring reasoning patterns as a portfolio of *named workflows*. A named workflow is a versioned orchestration contract rather than a runtime operator route or a sequence of prompts. At paper level, we represent it as

$$W_i = (\tau_i, I_i, \Delta_i, O_i, H_i, X_i, \Gamma_i),$$

where τ_i is an activation predicate, I_i and O_i are nominally typed input and output envelopes, Δ_i is an ordered set of artifact transformations, H_i is a set of control hooks, X_i is an exit gate, and Γ_i is an authority boundary. This makes explicit why a workflow starts, what it may transform, what

evidence it must leave, and which downstream workflow may receive its result. The type names currently constrain artifact identity and status within a common envelope; they do not yet provide a separate payload schema for every artifact class.

The portfolio has two layers. A small active layer supports routine target control, iterative search, proof-state maintenance, conceptual repair, release, and reporting. More expensive or specialized workflows activate only when their predicates hold. This preserves a lightweight default path. Catalog registration alone is insufficient: integration requires a declared workflow position, handoffs, hook enforcement, and an exit disposition.

5.1 Active lightweight workflows

Workflow	Conceptual transformation	Authority boundary
Target navigation	Open request → validated target, acceptance conditions, dependency frontier, and stopping rules.	Governs entry; does not establish a theorem or select a theory.
Discovery marathon	Controlled frontier → accumulated plural cycle evidence, retained failures, and next frontier.	Allocates exploration effort; grants neither proof nor truth authority.
Abducer work loop	Concrete obligations, negative results, and queue state → updated proof-state bundle.	Coordinates bounded attempts; cannot silently close proof gaps.
Conceptual hypothesis exploration	Structural failure → bridge objects, representation changes, claim revisions, and new validation obligations.	Produces hypotheses rather than validated claims.
Release promotion	Validation, dependency, compliance, and drift evidence → promote, quarantine, roll back, or escalate disposition.	Owns release disposition, not scientific truth.
Timeline paper	Accumulated evidence and conceptual change → claim-bounded historical manuscript.	Cannot modify theory state or upgrade claims.
Yang–Mills representational dynamics	Target state → observable, regulated, renormalization, infrared, continuum, and proof-facing packets.	Finite or regulated evidence does not imply continuum or mass-gap claims.

Table 1: Active workflows preserve a lightweight control path while separating coordination, scientific, and release authority.

These workflows do not form a mandatory line. Target navigation may invoke a discovery marathon or hand a concrete obligation directly to the work loop. Structural failure may activate conceptual exploration, whose candidate returns with new validation obligations. Release promotion is exceptional and applies only when durable state or official iteration status may change. Timeline writing is observational with respect to theory state, while the Yang–Mills programme is a target-specific overlay rather than a replacement for shared control semantics.

5.2 Opt-in and conditional workflows

The second layer houses transformations that would make routine navigation unnecessarily expensive. Activation therefore requires explicit evidence such as a source bundle, repeated conceptual blockage, an exact theorem statement, a result milestone, open manuscript modules, or a deep-review trigger.

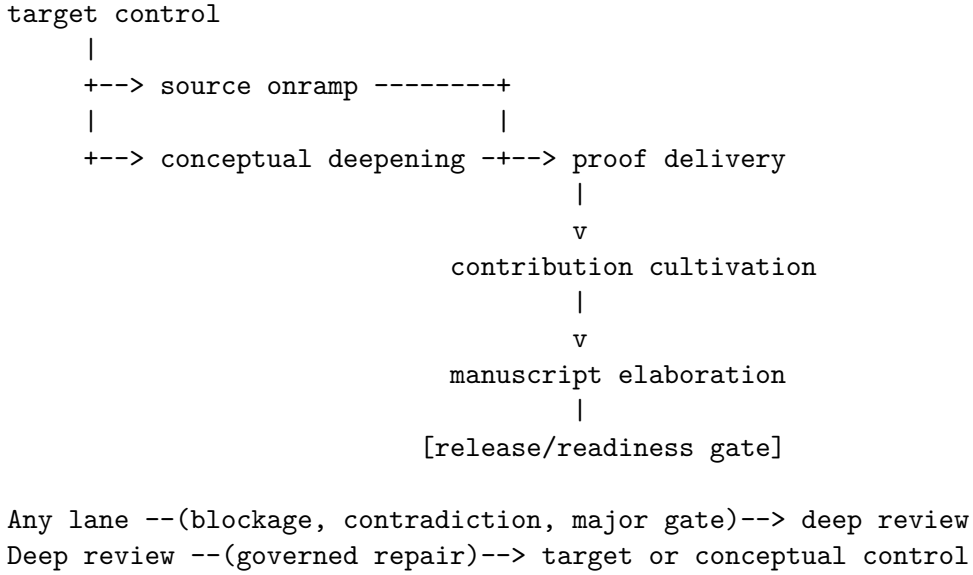
Workflow	Activation	Declared transformation	Exit meaning
Evidence-to-proof-route onramp	New sources, stale knowledge base, or proof-entrypoint request.	`SourceBundle` → normalized, grounded, debiased, falsification-tested `ExpertRouteDossier`.	Routes carry provenance, target implication, and falsifier or stop condition.
Conceptual frontier deepening	Frozen context, approved budget, and blockage, representation failure, or explicit aggressive search.	`FrozenDiscoveryContext` → plural conceptual systems → `FormalClaimProducerOutputV2`.	Claim is ready, rejected, or returned with a learning delta.
Theorem proof delivery	Exact, source- and target-bound theorem-shaped queue item.	`ProofQueueItem` → obligations and lemma chain → `ProofGateDecision`.	Proof candidate, conditional result, repair, countermodel, or rejection.
Scientific contribution cultivation	Result change, completed subgoal, contribution checkpoint, or paper-readiness request.	`ResultAtlasSlice` → value and target-enablement assessment → `EnablementAudit`.	Separates target-critical, reusable, local, side, and negative results.
Manuscript result elaboration	Draft with expansion markers, open modules, or elaboration request.	`PaperDraft` → bounded result packets → `PaperReadinessDecision`.	Integrates results while preserving finite, asymptotic, and continuum scopes.
Deep review and reorientation	Blockage, contradictory evidence, low explanatory power, major gate, or explicit review.	`RunEvidenceBundle` → advisory critiques and meta-review → `GovernedRepairQueue`.	Repair, pivot, quarantine, continuation, or stop; never proof acceptance.

Table 2: Opt-in and conditional contracts are declared escalation lanes rather than mandatory or necessarily executed stages.

5.3 Workflow schemas and conceptual roles

The following schemas provide a deliberately simple structural reading of the contracts. A box denotes an artifact-bearing transformation, a diamond-like `[gate]` denotes a decision that may stop, quarantine, reject, or hand off the run, and a branch denotes conditional rather than mandatory execution. The diagrams describe the intended control and information flow; they are not evidence that every component skill has executed or that a scientific claim is correct. Across all schemas, provenance, scope, open obligations, and epistemic status travel with the artifact rather than being inferred from its position in the route.

At portfolio level, the workflows form a graph of optional escalation lanes:



This is not a required pipeline. A request activates only the least expensive lane whose trigger and authority boundary fit the current obligation. Typed handoffs transport artifacts and unresolved obligations; they do not transfer proof, empirical, theory-selection, or release authority.

Evidence-to-proof-route onramp.

```
SourceBundle
   -> ingest / normalize / ground target
   -> connect proof routes
   -> audit hub bias and regime changes
   -> pressure with counterexamples
   -> [dossier gate]
         +--> ProofQueueItem -> theorem proof delivery
         `--> repair dossier -> conceptual control
```

Conceptually, this workflow is a provenance-preserving compiler from source material to a defensible research route. It prevents a salient citation or a high-degree knowledge-graph node from becoming a proof agenda merely by being easy to retrieve. Its output is a route dossier with falsifiers and stopping conditions, not a proof.

Conceptual frontier deepening.

```
FrozenDiscoveryContext + approved budget
   -> assimilate literature frontier
   -> generate and mature plural hypotheses
   -> reflect on complete conceptual systems
   -> blind, set-valued adjudication
   -> formalize surviving claim candidates
   -> [claim-readiness gate]
         +--> ProofQueueItem
         `--> ConceptualLearningDelta
```

This is the expensive theory-construction lane. Freezing context makes rival programmes comparable, independent maturation reduces premature convergence, and blind plural adjudication preserves underdetermination. Selection means "worthy of further formalization," never "true" or "proved."

Theorem proof delivery.

```
ProofQueueItem (exact statement, target, model, quantifiers)
   -> formalize proof obligations
   -> construct dependency-ordered lemma chain
   -> split gaps from countermodels and repairs
   -> audit inference and circularity
   -> [proof-quality gate]
         +--> accepted result packet -> contribution cultivation
         `--> gap/failure certificate -> conceptual control
```

This workflow turns a theorem-shaped proposal into an inspectable proof state. Its primary abstraction is an obligation graph: successful lemmas discharge nodes, while failed lemmas, circular dependencies, and countermodels remain first-class outcomes. Only the proof gate may issue a proof disposition, and that disposition remains distinct from novelty or publication readiness.

Scientific contribution cultivation.

```
ResultAtlasSlice
   -> isolate the smallest honest result
   -> check correctness boundary and prior art
   -> assess mechanism and reusable enablement
   -> align with target and paper claims
   -> [contribution gate]
         +--> contribution packet -> manuscript elaboration
         `--> bounded local/negative result or revision obligation
```

The conceptual purpose is valuation without inflation. A correct local lemma, a reusable method, a negative boundary, and a target-resolving result are different kinds of contribution. The workflow records those distinctions and requires mechanism evidence before a result is presented as enabling later work.

Manuscript result elaboration.

```
PaperDraft + explicit open modules
   -> turn markers into bounded obligations
   -> elaborate only supported result candidates
   -> certify theorem and regime scope
   -> audit prior-art disqualifiers
   -> integrate narrative with unresolved markers intact
   -> [paper-readiness gate]
         +--> bounded ready draft
         `--> proof queue, revision, quarantine, or rejection
```

This workflow is a controlled projection from validated scientific state into a manuscript. It may make an argument clearer or connect already supported results, but it may not perform hidden discovery or collapse finite, asymptotic, and continuum claims. Readiness therefore concerns faithful integration, not scientific truth by editorial fiat.

Route-A manuscript profile.

```
PaperDraft
   -> attack plan -> open-point plan
                     |
                     +--> specialist 1 --+
                     +--> specialist 2 --+--> typed result join
                     |        ...        |
                     `--> specialist n --+
                                 |
                        [same-theory / scope gate]
                                 |
                     optional symbolic-CAS audit
                                 |
                        integrate Route-A result
                                 |
```

```
        adapt ResultCandidatePacket to common validation tail
                                |
scope -> prior art -> narrative -> expansion -> readiness
```

Route-A is a profile, not an independent source of authority. Its fork permits specialists to attack distinct open points without sharing mutable intermediate state; the join admits only identity-, scope-, and provenance-compatible packets. The adapter then returns the joined result to the parent workflow's ordinary validation tail. Consequently, specialist agreement cannot bypass scope certification, prior-art review, or paper readiness.

Deep review and reorientation.

```
RunEvidenceBundle
   -> formalize current framework state
   -> independent advisory reviews
        {discovery, logic, epistemology, field, design}
   -> preserve agreement, dissent, and minority findings
   -> synthesize meta-review
   -> [repair-governance gate]
         +--> GovernedRepairQueue -> target/conceptual control
         `--> continue, quarantine, or stop
```

Deep review reasons about the adequacy of the process and its representation of the problem. Multiple perspectives are kept separate until synthesis so that consensus does not erase a decisive minority objection. The resulting queue may redirect work, but advisory agreement grants neither theorem status nor release acceptance.

The *evidence-to-proof-route onramp* turns source material into a defensible proof agenda by normalizing theorem bodies, grounding claims in the target conjecture, constructing proof connections, correcting graph-hub bias, auditing regime changes, and exposing routes to counterexample pressure. A successful dossier may become a proof queue item; a route needing repair returns to conceptual exploration.

Conceptual frontier deepening is the expensive theory-first lane. It freezes target, theory, and literature context before invention, matures hypotheses independently, reflects on complete conceptual systems, and performs blind plural adjudication. Experiments may constrain a later cycle but cannot select the theory that commissioned them. Conceptual preference licenses further formalization, not theorem status.

Theorem proof delivery starts only when source, target, quantifier, and model bindings are exact. It decomposes the claim into obligations and a lemma chain, makes blocked or false steps explicit, invokes target-specific inference review where applicable, and terminates in a proof-quality disposition. This contract organizes claim-to-proof or claim-to-gap work; it is distinct from the post-commit proof-v3 runtime, which alone constructs the descendant checker-scoped capability described later. Accepted result packets may enter contribution cultivation, while failure certificates return to conceptual work.

Scientific contribution cultivation distinguishes resolving the main target from obtaining an independently valuable result. It maintains a result atlas, identifies the smallest honest claim and prior-art boundary, and asks whether a completed subgoal enables later work. A paper handoff requires mechanism evidence and explicit unresolved boundaries.

Manuscript result elaboration operates on an existing draft without reopening the discovery marathon. It converts open modules into bounded obligations, certifies theorem scope, derives

finite-object or proof packets where needed, checks prior-art disqualifiers, and integrates validated results. Its optional *Route-A* profile specializes the reflected Wilson-block programme. Profile authorization orders planning before specialist work, maps the generic result-candidate object into the common validation-tail input, and then rejoins the parent scope, prior-art, narrative, expansion, and readiness gates. This is control-plane composition rather than evidence that the specialist result is mathematically valid.

Deep review and reorientation evaluates the reasoning system rather than the truth of its claims. It formalizes current framework state, collects several ordered advisory perspectives, synthesizes them, and produces a governed repair queue. It may redirect target navigation or conceptual exploration, but review consensus cannot establish proof, empirical support, release acceptance, or epistemic upgrade.

5.4 Hooks, handoffs, and non-monolithic composition

Each contract declares the lifecycle

$$\begin{aligned}
\textsc{Route} &: \text{request} \to \texttt{WorkflowRecommendation},\\
\textsc{Authorize} &: (W_i, \text{step}, \text{skill}, \text{context}) \to \texttt{SkillAuthorization},\\
\textsc{Receipt} &: (\text{output}, \text{status}, \text{evidence}) \to \texttt{SkillReceipt},\\
\textsc{ToolGate} &: (\text{tool}, \text{path}, \text{mutation}, \text{budget}) \to \texttt{ToolAuthorization},\\
\textsc{Exit} &: (O_i, \text{open obligations}) \to \texttt{WorkflowExitReceipt}.
\end{aligned}$$

Routing operates on structured trigger facts and is advisory: it recommends but does not activate a workflow or infer every trigger from free text. Pre-skill authorization checks workflow version, target, profile, step membership, prerequisites, and conditional evidence. Post-skill validation checks the nominal artifact type, presence, and epistemic status and emits a deterministic receipt. The stop hook applies the workflow-specific gate and open-obligation policy. These hooks govern admissibility and transport, not scientific merit; they cannot invent content, choose a theory, certify a theorem, reinterpret an experiment, or commit canonical state.

The tool hook binds tool use to the activated workflow version, target class, and step; requires a repository-relative target path, declared mutation class, and approved structured budget; and refuses deletion through the named-workflow policy. These operational checks remain separate from scientific authority and from resource accounting by an external execution platform.

A common but nonmandatory composition is

$$\begin{aligned}
\text{target navigation} &\to \{\text{source onramp},\ \text{discovery marathon}\}\\
&\to \{\text{conceptual exploration},\ \text{frontier deepening}\}\\
&\to \text{proof delivery} \to \text{contribution cultivation}\\
&\to \{\text{manuscript elaboration},\ \text{timeline paper}\}.
\end{aligned}$$

Deep review may intervene at a governed boundary, and release promotion is appended only when durable state must change. Failure edges are first-class: proof gaps return to conceptual work, routes may be quarantined, manuscript obligations may reopen proof delivery, and local contributions may be retained without claiming resolution of the main target. The portfolio therefore surrounds the AATS kernel with selectable declared lanes; it does not replace that kernel with a larger linear agent chain, demonstrate runtime transport merely by declaring a handoff, or transfer authority along handoff edges.

6 Semantic Feedback and Learning

An operator output is not evidence of learning merely because a file exists. For successful receipt r, let $\mathrm{Rows}(r)$ be the canonical semantic rows extracted from its artifact vector. The cycle requires a total map

$$\phi_r : \mathrm{Rows}(r) \to E_g$$

such that every row has exactly one digest-bound consumer event. It also records a role-specific primary consumer and a separate consumption marker. The feedback edge binds request and receipt identities, all artifact descriptors, the primary consumer, every pair $(q, \phi_r(q))$, and committed generation $g + 1$. Disconnected but individually valid artifacts are rejected from the transition.

Negative results are first-class. Decisive contradictory evidence can refute and retire a hypothesis while preserving its genealogy and the premise that failed. A literature refresh may stale a branch without erasing the earlier state. Thus cumulative learning includes constraints on future search, not only accepted positive claims.

7 Authority Separation

We distinguish *write capability*, permission to mutate canonical state, from *epistemic authority*, evidence licensing a scientific status. Let Γ contain capabilities for proposal, conceptual preference, claim provenance, checker result, empirical observation, source truth, theorem status, external validation, physics result, publication integrity, and canonical state write. For artifact x,

$$\mathrm{Auth}(x) \subseteq \Gamma \times \mathcal{E}_{\mathrm{auth}}$$

contains only capability–evidence pairs licensed by a capability-specific validator. The coordinates are mutually incomparable: evidence for one capability creates no order-theoretic promotion into another.

Design invariant 1 (Non-escalation)**.** *No transition may infer proof, theorem, empirical, source-truth, physics-result, or external-validation authority solely from conceptual preference, operator success, schema validity, or canonical persistence.*

This is the central epistemic abstraction: different institutions answer different questions. Generators propose; adjudicators express internal conceptual preference; validators check contracts; proof systems record checker-scoped results under declared assumptions; experiment artifacts encode observations against registered premises; the reducer records authorized effects. Publication is a distinct integrity and acceptance capability and does not upgrade scientific claims.

8 Proof and Experiment as Descendant Lanes

Proof and experiment are subordinate to the theory transaction rather than coequal theory selectors.

8.1 Formal claim and proof lane

A registered producer may compile a typed formal-claim packet while assembling the theory transaction, but that packet has no proof-launch capability. Only after its request, receipt, input, registry, immutable output history, semantic consumer event, and feedback edge have been committed and

Institution	Permitted effect	Explicit non-authority
Proposal operators	Produce typed candidates	Preference, proof, empirical truth, state write
Conceptual panel	Return an internal plural preference	Theorem, external validation, source truth
Reducer/transaction	Commit validated semantic state	Scientific truth from persistence alone
Proof-v3	Record checker-scoped outcomes	Checker soundness, physics, parent-theory selection
Experiment descendant	Produce later-intake observation records	Same-generation mutation, proof, physics result
Publisher	Select byte-exact accepted bundles	Novelty, efficacy, theorem or source truth

Table 3: Authority is capability-specific and non-escalating.

reopened through one scheduled prove action may proof-v3 construct a live one-shot capability. A successful worker records a checker-scoped result under declared assumptions; checker soundness, theorem truth, physical relevance, and parent-theory selection remain null.

8.2 Delayed experimental feedback

Same-generation experiment artifacts are forbidden from mutating the theory that commissioned them. The descendant route has the chronology

$$N \to N+1 \to \{N+2, \ldots, N+6\} \to N+7 \to N+8,$$

where N through $N+8$ are consecutive transaction-store generations, not eight theory iterations. Generation $N+1$ commits the world-matrix anchor. Five world products may be computed concurrently, but one coordinator commits them in fixed order at $N+2$ through $N+6$; $N+7$ commits their aggregation and $N+8$ the sole immutable descendant-reflection root. The worlds include no probe, neutral or inconclusive evidence, a misleading toy positive, a registered-premise falsifier, and operational failure.

Design invariant 2 (Delayed intervention)**.** *No descendant-lane artifact has a transition edge into S_N. Any later theory effect requires a distinct authority-checked consumer transition citing the $N+8$ root.*

The delay is a control invariant, not evidence that eight generations is empirically optimal or that the design eliminates self-confirming bias. The $N+8$ root is only eligible input to a later theory cycle; it has no automatic theory effect.

9 Persistence and Concurrency

The logical ledger is an ordered, hash-chained event stream, while the transaction store snapshots reduced state and per-commit event suffixes. Reduction accepts only a suffix extending the exact predecessor hash and generation, applies semantic deduplication, and deterministically reconstructs the next state.

The transaction store stages state and artifacts, flushes bytes, records a durability journal, atomically installs a content- and request-bound immutable commit directory, and then updates

a small pointer. Within one target/run store, non-idempotent writers carrying the same expected generation cannot install distinct successors; an exact retry may return the installed commit.

Design invariant 3 (Isolation)**.** *Workers using the parallel API receive no canonical-state capability; the merge boundary alone receives the optional CAS callback.*

Design invariant 4 (Atomicity)**.** *Within one target/run transaction store, at most one non-idempotent transition from an expected generation wins compare-and-swap.*

Design invariant 5 (Replay)**.** *For a fixed reducer version, exact validated predecessor, and byte-identical ordered event suffix, reduction reconstructs the same state bytes and semantic digest.*

Design invariant 6 (Plurality)**.** *Adjudication may preserve several nondominated conceptual programmes and their dissent rather than collapsing them to a scalar winner.*

10 Executable Realization

The AATS core is realized through four main surfaces, with a cross-cutting named-workflow control plane:

Role	Repository surface	Responsibility
State and reduction	`agentic/discovery/ledger.py`	Typed state, event algebra, negative memory, deterministic replay
Cycle composition	`agentic/discovery/cycle.py`	Identity checks, semantic consumption, authority gates, single transition
Operators	`agentic/registries/discovery_operators.py`	Versioned registry of framers, validators, effects, typed-only routes
Persistence	`agentic/state/discovery_transaction.py`	Immutable commits, journals, recovery, content identity, CAS
Workflow composition	Named-workflow registries, artifact envelope, hooks, and validator	Versioned activation, step membership, declared handoffs, receipts, and exit gates

Table 4: Concrete realization of the authority-separated abductive transition system.

The canonical cycle consumes digest-verified artifacts for one target, run, and frozen generation. Required roles include literature snapshot and production input, assimilation, high-risk portfolio, parallel portfolio, and reflective packets. Literature-conditioned invention and central conceptual v3 are typed proposal subroutes without direct preference, proof, or state-write authority. Conditional formal claims join post-commit proof-v3; descendant experiments use a separate transaction chain.

11 Validation Posture

We executed a targeted conformance matrix on 5 August 2026 using the locked `uv` environment. Transaction, ledger, parallel-portfolio, and conceptual-adjudication suites completed 127 cases successfully; the canonical proof-loop suite completed six, for 133 fresh component passes. The descendant-runtime command did not produce a complete pytest summary and is therefore not counted as fresh conformance evidence. Separately, both canonical acceptance cases failed, which is reported rather than averaged into the passing component evidence.

Route	Current role	Authority boundary
Parent theory transaction	Active reducer contract	One validated semantic CAS transition
Central conceptual v3	Active typed-only proposal route	No direct state write or merit authority
Literature-conditioned invention v1	Registered proposal route	No truth, creativity, or theory-selection authority
Proof-v3	Active post-commit descendant	Checker-scoped result; no parent mutation
Experiment v3	Active N–$N+8$ descendant	Later-intake root only
Historical six-operator publication	Legacy evidence	Rejected by current publication/acceptance surface

Table 5: Current and historical workflow surfaces must not be conflated.

Question	Adversarial condition	Enforced invariant	Fresh observation
Concurrency	stale competing writer	one per-run CAS successor	component suite passed
Semantic use	valid but unconsumed output	no canonical state effect	component suite passed
Plural search	partial branch failure	valid siblings and dissent retained	component suite passed
Replay	exact predecessor/event suffix	same state bytes and semantic digest	component suite passed
Authority	preferred theory seeks proof status	no cross-capability escalation	component suite passed
Experiment	same-generation mutation attempt	descendant-only later intake	contract/test-backed; fresh suite incomplete
Release	checked-in current publication	v2 pointer and current topology	failed: legacy pointer
Negative fixture	local-only literature case	reach intended literature gate	failed earlier: empty old chain

Table 6: Conformance evidence. Passing component tests do not imply scientific efficacy or end-to-end release acceptance.

They do not show that Abducer discovers better theories, solves open problems, improves human productivity, calibrates truth, or generalizes beyond the tested contracts. At the time of writing, the checked-in canonical publication uses a legacy index rejected by the current pointer loader, and the release acceptance fixture still expects an older same-generation experiment topology. Consequently, historical six-operator artifacts must not be presented as current end-to-end acceptance evidence. The current abstraction is a theory transaction followed by descendant proof and experiment support.

Named-workflow control-plane evidence is separate from those core acceptance results. On 5 August 2026, 30 focused contract-registry, projection, hook, and scenario tests passed. Seven positive lifecycles cover every named workflow plus Route-A, traversing 71 activated step or specialist positions with realistic domain fixtures; negative cases cover absent activation, prohibited triggers, premature exit, cross-target receipt tampering, bypassed advisory review, premature specialist use, deletion, and unapproved budgets. Artifact receipts are bound to workflow version, target, producer, path, and content digest, while explicit adapters make non-direct handoffs inspectable. The six named contracts directly compose 75 unique skills; the wider integration report classifies 218 catalogued skills as workflow-declared only by taking the union of named workflows, legacy workflows and tasks, dispatch, operator routes, and other explicit dispositions. That report identifies eight skills as runtime-routed and executable, and none as scientifically exercised, semantically consumed, or feedback-effective. The scenario suite exercises control transport using fixture envelopes; it does

not run all component skills, execute downstream handoff adapters, validate a schema specific to each payload type, or demonstrate scientific outcomes.

12 Related Work Evidence Status

No provenance-valid external literature snapshot was authorized for this repository-grounded paper. We therefore do not claim novelty or comparative advantage relative to abductive reasoning systems, LLM scientific agents, theorem-proving agents, blackboard or event-sourced architectures, workflow-provenance systems, or plural model-selection methods. A submission-oriented version requires an independently governed literature search and claim-to-source mapping across those areas.

13 Limitations

First, this is a repository-grounded systems description without an authorized external literature review; no claim of novelty relative to other agentic discovery, scientific-workflow, theorem-proving, or event-sourced systems is made. Second, internal judges are authority-distinct programs but not independent human experts. Third, contract validity and replay do not imply mathematical soundness or scientific value. Fourth, the quality of the transition depends on the semantic content supplied by generative operators, while the present evidence primarily establishes control invariants. Fifth, the legacy KB loop is only partially executable because abduction and deduction require manual prompt-driven interaction. Sixth, named-workflow artifacts retain nominal rather than payload-specific types, adapter declarations and fixture transport are not evidence of live downstream skill execution, and structured budgets are authorization records rather than measured resource enforcement. Seventh, the current canonical publication and acceptance surfaces require reconciliation before a fresh run can serve as release evidence.

14 Research Agenda

The abstraction suggests a concrete evaluation programme: compare scalar and set-valued selection; ablate negative-memory retention; test whether delayed feedback reduces self-confirming revisions; measure whether semantic-consumption edges improve audit accuracy; and conduct held-out, blinded expert evaluation of conceptual relevance and understanding change. Such studies require frozen tasks, preregistered metrics, independently governed evidence, and explicit separation between protocol conformance and scientific efficacy.

15 Conclusion

Abducer operationalizes abduction as controlled evolution of a plural epistemic state. Its essential mechanism is not a sequence of prompts, but the composition of diverse proposal, isolated maturation, recursive transformation, set-valued adjudication, semantic feedback, capability-specific authority, deterministic reduction, and atomic persistence. Around that kernel, named workflows organize source intake, conceptual deepening, proof delivery, contribution assessment, manuscript elaboration, reorientation, and target-specific work as optional, declared escalation lanes. Proof and experiment produce authority-scoped descendant records eligible for later intake without inheriting theory-selection authority, and workflow handoffs likewise transport no authority by themselves.

This formulation makes the system precise enough to inspect, replay, falsify at its contract boundaries, and evaluate without confusing generated ideas, declared integration, executed transport, and established knowledge.

A Repository Evidence Map

The principal local evidence for this description is:

- `agentic/spec/discovery_epistemic_state.schema.json`: state ontology and identity restrictions;
- `agentic/registries/discovery_operator_registry.yaml`: operator, consumer, effect, and authority contracts;
- `agentic/discovery/cycle.py`: canonical composition and semantic feedback;
- `agentic/discovery/ledger.py`: event reduction, literature rebase, refutation, and negative memory;
- `agentic/discovery/parallel.py`: isolated sibling execution and single-writer merge;
- `agentic/discovery/evaluation.py`: noncompensatory conceptual gates;
- `agentic/discovery/conceptual_adjudication.py`: semantic panel and set-valued preference;
- `agentic/discovery/blind_conceptual_judge_v3.py`: blind aliasing and Pareto reduction;
- `agentic/discovery/literature_conditioned_invention.py`: registered proposal-generation route;
- `agentic/discovery/scheduler.py`: Pareto action scheduling and protected wildcards;
- `agentic/state/discovery_transaction.py`: immutable commits, journals, recovery, and CAS;
- `agentic/discovery/proof_v3_facade.py`: post-commit proof execution boundary;
- `agentic/experiments/world_v3_content_graph_runtime.py`: descendant experiment realization;
- `agentic/registries/named_workflow_contracts.yaml`: versioned named-workflow and Route-A profile declarations;
- `agentic/spec/named_workflow_artifacts.schema.json`: shared nominal artifact envelope;
- `agentic/reports/skill_integration/latest.json`: declared and runtime integration classifications;
- `registry/tasks/named-skill-workflows/task_registry.json`: generated task projection;
- `hooks/scripts`: advisory routing, workflow-local skill and output checks, tool guard, and exit gate;
- `skills/orchestration/reflective-conceptual-discovery/SKILL.md`: theory-first workflow contract;
- `docs/research-methods/discovery-operator-routes.md`: canonical route and chronology guide.